

IV + Greeks Engine Performance Benchmark Report

This report analyzes the benchmark and latency-distribution performance of the header-only C++ Implied Volatility and Greeks analytics engine. The benchmark includes: 100,000 option calculations Implied volatility convergence Greeks computation Newton-Raphson iterative solving Tail latency percentile analysis

1. Benchmark Results

Metric	Value
Options Processed	100,000
Total Time	43.0724 ms
Average Latency	0.3395 μ s
Throughput	2,945,508 options/sec
IV Checksum	29333.2

The engine demonstrates extremely high throughput for a single-threaded option analytics utility. Achieving nearly 3 million option calculations per second indicates: Low-overhead execution Efficient inline computation Minimal branching overhead Cache-friendly architecture Stable convergence behavior

2. Latency Distribution Analysis

Percentile	Latency
Minimum	100 ns
p50	400 ns
p90	600 ns
p95	600 ns
p99	800 ns
p99.9	900 ns
Maximum	118,000 ns (118 μ s)

The latency distribution is exceptionally tight. Key observations: 99% of all computations complete under 1 microsecond. The p50-to-p99 spread is only 2x, indicating highly stable execution. Very low tail jitter suggests predictable convergence behavior. The engine shows deterministic runtime characteristics. The isolated 118 μ s maximum latency spike is most likely caused by: OS scheduler interruptions Context switching CPU frequency transitions Background system activity Such spikes are normal in microbenchmark environments.

3. Engine Architecture Assessment

The engine architecture demonstrates characteristics commonly found in low-latency quantitative systems. Key architectural strengths: Header-only design Inline execution path Allocation-free computation Branch-light control flow Minimal cache pressure Stable Newton-Raphson convergence These characteristics are highly desirable in: Risk Management Systems (RMS) Real-time Greeks servers Options chain calculators Market-making systems Quantitative pricing engines

4. Current Performance Bottlenecks

At the current performance level, the primary runtime bottlenecks are no longer the application architecture itself. The dominant cost now comes from mathematical transcendental functions: `std::exp` `std::log` `std::sqrt` These operations are CPU-intensive and dominate execution time in Black-Scholes and Black-76 pricing models.

5. Recommended Next Steps

Recommended next-level enhancements: SIMD vectorization using AVX2/AVX512 Structure-of-Arrays (SoA) memory layout Parallel multi-core execution Near-expiry stress testing Real NSE option chain replay testing Iteration percentile tracking CPU affinity benchmarking Cycle-level `rdtsc` benchmarking Potential future throughput with SIMD + multithreading: 10M+ options/sec achievable Sub-500ns p99 latency possible

6. Final Assessment

The benchmark results indicate that the current implementation is already suitable for production-grade real-time option analytics workloads. The most important achievement is not only raw throughput, but the extremely stable latency distribution. A p99 latency below 1 microsecond is a strong indicator that the engine is: Deterministic Numerically stable Cache efficient Suitable for low-latency systems Overall Assessment: Performance Quality: Excellent Tail Latency Stability: Excellent Scalability Potential: Very High Production Readiness: Strong Foundation